

# integration\_sec03

## 3. Phase 2: Core Verification Engine Integration

Phase 2 forms the operational heart of the LLMsVerifier integration by establishing the full verification engine within the HelixTranslate service boundary. Where Phase 1 concentrated on configuration scaffolding and service discovery, Phase 2 moves into implementation territory: constructing the Go-based verifier client, defining the eight-step pipeline architecture, implementing each verification stage from existence checks through final validation, and wiring the entire apparatus into the existing HelixTranslate startup sequence. The deliverables of this phase are concrete, production-hardened Go source files that reside in `internal/verifier/` and are imported by `cmd/helix-translate/main.go`. Every component introduced in this chapter is designed with observability, fault tolerance, and horizontal scalability as first-class concerns. The design assumes that the verifier may reside on a separate host, may experience transient failures under load, and must not block the translation hot path for longer than strictly necessary.

The architectural philosophy guiding this phase is one of defensive composition: each verification step is an isolated unit that exposes a common `Step` interface, participates in a group-based execution strategy (parallel versus sequential), carries its own timeout budget, and reports success or failure through a uniform `StepResult` structure. Steps that are marked as critical can halt the entire pipeline and prevent a model from entering the active pool, while non-critical steps merely contribute to a composite score. This distinction allows HelixTranslate to adopt a graduated gating strategy—models must pass all critical checks but can be admitted with warnings on secondary dimensions such as advanced feature availability or cost competitiveness. The pipeline is orchestrated by a `VerificationPipeline` struct that uses Go's `errgroup` package for concurrent step execution within Group 1, followed by a straightforward sequential loop for Group 2. Retry logic with exponential backoff and circuit-breaker awareness protects against transient network failures and thundering-herd scenarios when multiple verification cycles are triggered simultaneously.

Throughout this chapter, references to file paths assume the standard Go project layout: domain logic lives under `internal/verifier/`, configuration structures are defined in `internal/config/`, shared cache abstractions reside in `internal/cache/`, and the service entry point is `cmd/helix-translate/main.go`. All timeout values, retry parameters, and threshold constants presented here are defaults that can be overridden through the `LLMsVerifierConfig` structure documented in Phase 1. The implementation also respects context cancellation, ensuring that a

shutdown signal propagates cleanly through long-running verification steps and causes the pipeline to return immediately rather than hanging on an outbound HTTP call.

---

## 3.1 Verifier Client Initialization

The first concrete task in Phase 2 is to create the `VerifierClient` struct and its constructor, which encapsulates all communication between `HelixTranslate` and the `LLMsVerifier` backend service. This client is intentionally thin: it is responsible for connection management, request signing via JWT bearer tokens, and response deserialization, but it delegates all business logic to the pipeline layer above it. Keeping the client thin simplifies unit testing—mocking the verifier backend requires only a small HTTP test server—and reduces the blast radius of any API contract changes on the verifier side.

The client is defined in `internal/verifier/client.go`. It holds four fields: an `*http.Client` instance configured with a timeout derived from the global configuration, a `baseURL` string representing the verifier service endpoint, an `apiKey` used for Bearer-token authentication in every outbound request, and a `timeout` field cached locally for diagnostic logging. The constructor `NewVerifierClient` performs strict validation: both `APIURL` and `APIKey` are required fields, and the function returns an explicit error rather than silently substituting defaults for values that represent security-sensitive or connectivity-critical configuration. The timeout is sourced from `cfg.CacheTTL` as a sensible default, but falls back to thirty seconds if that value is zero, ensuring that the client is usable even in minimally configured deployments.

### CODE BLOCK 1: `internal/verifier/client.go`

```
type VerifierClient struct {
    httpClient *http.Client
    baseURL    string
    apiKey     string
    timeout    time.Duration
}

func NewVerifierClient(cfg *config.LLMsVerifierConfig)
    (*VerifierClient, error) {
    if cfg.APIURL == "" { return nil, errors.New("APIURL is
        required") }
    if cfg.APIKey == "" { return nil, errors.New("APIKey is
        required") }
    timeout := cfg.CacheTTL
    if timeout == 0 { timeout = 30 * time.Second }
    return &VerifierClient{
        httpClient: &http.Client{Timeout: timeout},
        baseURL:    strings.TrimRight(cfg.APIURL, "/"),
        apiKey:     cfg.APIKey,
        timeout:    timeout,
    }, nil
}
```

```

func (c *VerifierClient) Ping(ctx context.Context) error {
    req, err := http.NewRequestWithContext(ctx, "GET",
        c.baseURL+"/health", nil)
    if err != nil { return err }
    req.Header.Set("Authorization", "Bearer "+c.apiKey)
    resp, err := c.httpClient.Do(req)
    if err != nil { return err }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK { return
        fmt.Errorf("verifier unhealthy: %d", resp.StatusCode) }
    return nil
}

```

The Ping method is the simplest possible health-check: it sends a GET request to the verifier's /health endpoint, injects the Bearer token into the Authorization header, and validates that the response status is exactly 200 OK. This method is intentionally lightweight so that it can be called frequently during startup and during periodic liveness checks without placing meaningful load on either service. The method respects context cancellation, meaning that if the parent context expires or is cancelled, the outbound HTTP request will abort immediately rather than continuing to occupy a connection slot.

During service startup in cmd/helix-translate/main.go, the client initialization must occur after configuration parsing but before the translation service begins accepting traffic. The recommended startup sequence is: parse flags and environment variables into config.LLMsVerifierConfig; invoke NewVerifierClient(cfg) and check the returned error, logging a Fatal if either required field is missing; then call WaitForHealthy (defined below) with a context bounded by a configurable maximum wait duration, typically thirty to sixty seconds. Only after the verifier reports a healthy status does the service proceed to initialize its model pool and start the HTTP or gRPC server. This ordering guarantees that no translation request can arrive before the verification subsystem is operational.

Beyond basic connectivity, the client struct serves as the single point of configuration for HTTP connection pooling. The http.Client instantiated by the constructor uses Go's default transport, which already supports persistent connections and connection reuse. For high-throughput deployments, operators may wish to customize the transport with MaxIdleConns, MaxIdleConnsPerHost, and IdleConnTimeout values. Because the transport is created inside the constructor, this customization can be exposed through additional fields in LLMsVerifierConfig without changing the call sites of NewVerifierClient. The key design invariant is that every outbound request to the verifier service is made through this single client instance, ensuring consistent timeout behavior and authentication across all verification operations.

## CODE BLOCK 2: internal/verifier/health.go

```

func (c *VerifierClient) WaitForHealthy(ctx context.Context,
    maxWait time.Duration) error {
    deadline := time.Now().Add(maxWait)
    for time.Now().Before(deadline) {
        if err := c.Ping(ctx); err == nil { return nil }
        time.Sleep(time.Second)
    }
}

```

```

    return fmt.Errorf("verifier not healthy after %v", maxWait)
}

```

The `WaitForHealthy` helper complements `Ping` by providing a blocking poll loop with a hard deadline. It is intended for use exclusively during startup, when `HelixTranslate` must delay its own readiness until the verifier is available. The loop polls once per second, a cadence chosen to be frequent enough to minimize startup latency without generating excessive log noise or network traffic. The deadline is computed from `maxWait`, which should be provided from configuration (e.g., `cfg.StartupMaxWait`) so that operators can tune it based on their deployment topology. If the verifier does not become healthy within the allotted window, the function returns a descriptive error that should be treated as fatal by the caller: attempting to operate a translation service without a functioning verification backend violates the safety guarantees of the integrated system.

---

## 3.2 The 8-Step Verification Pipeline

With the client initialized, the next deliverable is the pipeline orchestrator that sequences the eight verification steps. The pipeline is the central abstraction of Phase 2: it receives a model identifier, coordinates the execution of all verification handlers, aggregates their individual results into a composite report, and makes the final admission decision. The design splits the eight steps into two execution groups based on dependency constraints and latency characteristics.

Group 1 contains the first four steps—Existence, Responsiveness, Feature Detection, and Code Visibility. These steps are independent of one another and are relatively fast (timeouts ranging from five to twenty seconds). They are executed in parallel using an `errgroup.Group`, which both bounds concurrency and provides unified error propagation: if any critical step in Group 1 fails, the group context is cancelled and the remaining goroutines receive the cancellation signal, preventing wasted work. Group 2 contains the final four steps—Coding Challenge, Performance, Cost Analysis, and Final Validation. These steps must run sequentially because each step may depend on the results or side effects of its predecessor. For example, the Performance benchmark may use a model configuration discovered during Feature Detection, and Final Validation cannot execute until all prior results are available.

**TABLE 1: Pipeline Step Mapping**

| Step | Name              | Handler File                         | Timeout | Critical | Parallel Group |
|------|-------------------|--------------------------------------|---------|----------|----------------|
| 1    | Existence         | internal/verifier/existence.go       | 5s      | Yes      | Group 1        |
| 2    | Responsiveness    | internal/verifier/responsive.go      | 10s     | Yes      | Group 1        |
| 3    | Feature Detection | internal/verifier/feature_detect.go  | 15s     | No       | Group 1        |
| 4    | Code Visibility   | internal/verifier/code_visibility.go | 20s     | No       | Group 1        |
| 5    |                   |                                      | 60s     | No       | Group 2        |

| Step | Name             | Handler File                          | Timeout | Critical | Parallel Group |
|------|------------------|---------------------------------------|---------|----------|----------------|
|      | Coding Challenge | internal/verifier/coding_challenge.go |         |          |                |
| 6    | Performance      | internal/verifier/perf.go             | 120s    | No       | Group 2        |
| 7    | Cost Analysis    | internal/verifier/cost.go             | 10s     | No       | Group 2        |
| 8    | Final Validation | internal/verifier/validation.go       | 5s      | Yes      | Group 2        |

Each step in the pipeline implements the Step interface, which normalizes interaction between the orchestrator and the individual handlers regardless of their internal complexity.

### CODE BLOCK 3: Pipeline with Parallel Groups

```

type Step interface {
    Name() string
    Execute(ctx context.Context, modelID string) (*StepResult,
        error)
    Timeout() time.Duration
    IsCritical() bool
}

type VerificationPipeline struct {
    group1 []Step // parallel: existence, responsive, feature,
        code
    group2 []Step // sequential: coding, perf, cost, validation
}

func (p *VerificationPipeline) Run(ctx context.Context, modelID
    string) (*PipelineResult, error) {
    g, ctx := errgroup.WithContext(ctx)
    results := make([]*StepResult, 0, 8)

    // Group 1: parallel
    for _, step := range p.group1 {
        s := step
        g.Go(func() error { r, err := s.Execute(ctx, modelID);
            results = append(results, r); return err })
    }
    if err := g.Wait(); err != nil { return nil, err }

    // Group 2: sequential
    for _, step := range p.group2 {
        r, err := step.Execute(ctx, modelID)
        results = append(results, r)
        if err != nil && step.IsCritical() { return nil, err }
    }
    return aggregateResults(results), nil
}

```

The `Step` interface enforces four methods. `Name()` returns a human-readable identifier used for logging, metrics tagging, and error messages. `Execute()` performs the actual verification work against a specific model identifier and returns a `*StepResult` containing a boolean `Passed` field, structured output data, and timing metadata. `Timeout()` declares the maximum duration the step is allowed to run before its context is cancelled. `IsCritical()` determines whether a failure in this step should cause the entire pipeline to abort. This last attribute is what enables the graduated admission policy: Existence and Responsiveness are critical because a model that does not exist or cannot respond is fundamentally unusable, while Feature Detection and Code Visibility are non-critical because their absence only limits functionality rather than preventing basic operation.

The `VerificationPipeline.Run` method is the engine's entry point. It begins by creating an `errgroup.WithContext(ctx)`, which derives a child context that is cancelled if any goroutine in the group returns a non-nil error. This is essential for the Group 1 parallel execution: if the Existence check fails critically, the derived context is cancelled, causing in-flight Feature Detection and Code Visibility calls to abort early. After Group 1 completes (either successfully or with a critical failure), Group 2 executes in a simple for loop. Each step's result is appended to the shared results slice, and critical failures immediately halt further progress. The final call to `aggregateResults` computes the `PipelineResult`, including the overall pass/fail determination and a summary of per-step outcomes.

Individual steps, particularly those that make network calls to external model APIs, are subject to transient failures. To prevent a single timeout or 503 Service Unavailable from causing a model to be incorrectly rejected, each step's `Execute` method should be wrapped in a retry decorator. The retry logic uses exponential backoff with jitter to avoid synchronized retry storms, and it respects the circuit-breaker pattern by tracking recent failure rates and temporarily bypassing steps that have exceeded a failure threshold.

#### CODE BLOCK 4: Retry with Circuit Breaker

```
func executeStepWithRetry(step Step, ctx context.Context, modelID
    string, cfg RetryConfig) (*StepResult, error) {
    var lastErr error
    for attempt := 0; attempt < cfg.MaxAttempts; attempt++ {
        if attempt > 0 { time.Sleep(cfg.Backoff *
            time.Duration(1<<attempt)) }
        result, err := step.Execute(ctx, modelID)
        if err == nil { return result, nil }
        lastErr = err
        if step.IsCritical() && attempt < cfg.MaxAttempts-1 {
            continue }
    }
    return nil, fmt.Errorf("step %s failed after %d attempts:
        %w", step.Name(), cfg.MaxAttempts, lastErr)
}
```

The `executeStepWithRetry` function applies a configurable number of retry attempts with exponential backoff. The backoff calculation uses `cfg.Backoff * time.Duration(1<<attempt)`, producing delays that double on each subsequent retry: if the base backoff is one second, the delays are one second, two seconds, and four seconds for attempts two, three, and four respectively. This aggressive backoff prevents the verifier client from hammering an already-stressed upstream API. The

loop continues only for critical steps: if a non-critical step fails on every attempt, the function returns the final error but does not propagate it as a pipeline-halting failure. Instead, the caller records the step as failed but allows subsequent steps to proceed. For critical steps, the loop exhausts all retries before returning the wrapped error, which `VerificationPipeline.Run` then treats as a reason to abort the entire pipeline. The circuit-breaker integration, while not shown inline here, is realized by checking a `CircuitBreaker.Allow()` predicate at the start of `Execute`; if the breaker is open, the method returns immediately with a sentinel error that bypasses the retry loop.

---

### 3.3 Existence and Responsiveness Verification

The first two pipeline steps establish the most fundamental prerequisites: that the model identifier corresponds to a real, queryable model in the target provider's catalog, and that the model's inference endpoint responds within acceptable latency. These checks are both critical and fast, making them ideal candidates for Group 1 parallel execution.

Existence verification queries the model provider's API (or the verifier service's cached model registry) to confirm that the model ID is valid and the model is not deprecated or access-restricted. Because model catalogs change infrequently, the existence check benefits heavily from caching. A positive result is cached for five minutes, while a negative result is cached for thirty seconds to prevent rapid re-querying of a model that is legitimately unavailable. The cache key is a composite string formed as `"existence:" + modelID`, namespaced to avoid collisions with other cached verification data.

#### CODE BLOCK 5: Existence and Responsiveness with Caching

```
func (v *Verifier) VerifyModelExistence(ctx context.Context,
    modelID string) (bool, error) {
    if cached, ok := v.cache.Get("existence", modelID); ok {
        return cached.(bool), nil
    }
    exists, err := v.client.CheckModelExists(ctx, modelID)
    if err == nil { v.cache.Set("existence", modelID, exists,
        5*time.Minute) }
    return exists, err
}
```

The `VerifyModelExistence` method begins by consulting the cache abstraction injected into the `Verifier` struct at construction time. The cache interface is defined in `internal/cache/` and supports pluggable backends (in-memory LRU for single-instance deployments, Redis for replicated deployments). If a cached value is found, it is returned immediately without any network call, reducing the latency of the Existence step to microseconds. On a cache miss, the method delegates to `v.client.CheckModelExists`, which performs an HTTP HEAD or GET request against the verifier service's model registry endpoint. The result is written back to the cache only on success, ensuring that transient errors do not poison the cache with false negatives.



Responsiveness verification complements existence by measuring the round-trip time to the model's inference endpoint. This step does not perform a full translation; instead, it sends a minimal prompt (such as a single-word echo request) and validates that a response is received within the ten-second timeout. The measured latency is recorded in the `StepResult` and contributes to the composite health score, but the step's criticality is based solely on whether any response is received at all. A model that exists but cannot serve inference within the timeout window is treated as unavailable and is not admitted to the active pool. Like existence, responsiveness results are cached, but with a shorter TTL of one minute to ensure that the system reacts promptly to restored connectivity.

Together, these two steps form the "gatekeeping" layer of the verification pipeline. They are the fastest steps but carry the highest stakes: a failure here means the model is either phantom or unreachable, and no amount of feature richness or performance benchmarking can compensate. The caching strategy is particularly important at scale, where hundreds of models may be verified in a tight loop during a provider catalog refresh. Without caching, the system would generate a thundering herd of existence queries against the verifier service; with caching, only the first verification of each model in a five-minute window incurs network overhead.

---

## 3.4 Feature Detection and Code Visibility

Once a model's existence and basic responsiveness are confirmed, the pipeline proceeds to assess its functional capabilities. Feature Detection enumerates which LLM capabilities are supported by the target model, while Code Visibility evaluates the model's effectiveness with programming-language content. These two steps are non-critical because HelixTranslate can operate in a degraded mode when advanced features are unavailable, but their results are essential for routing decisions and for presenting accurate capability metadata to downstream consumers.

Feature detection uses a capability bitmap approach, which is both compact and efficient for comparison operations. Each capability is assigned a bit position in a `uint32` mask, allowing the full feature set of a model to be represented as a single integer. This representation enables fast subset queries—for example, checking whether a model supports both streaming and function calling is a single bitwise AND operation rather than a map lookup or string comparison.

### CODE BLOCK 6: Feature Detection

```
type CapabilityBitmap uint32
const (
    CapStreaming CapabilityBitmap = 1 << iota
    CapFunctionCalling
    CapVision
    CapCodeGeneration
    CapJSONMode
    CapSystemPrompts
    CapLargeContext
    CapMultilingual
)
```



```

func DetectFeatures(ctx context.Context, client VerifierClient,
    modelID string) (CapabilityBitmap, error) {
    caps := CapabilityBitmap(0)
    if hasStreaming(ctx, client, modelID) { caps |= CapStreaming }
    if hasFunctions(ctx, client, modelID) { caps |=
        CapFunctionCalling }
    if hasVision(ctx, client, modelID) { caps |= CapVision }
    return caps, nil
}

```

The CapabilityBitmap type defines eight capability flags at present, with the most significant bit positions reserved for future expansion. The DetectFeatures function initializes an empty bitmap and then probes each capability independently. Each probe function (hasStreaming, hasFunctions, hasVision) sends a targeted test request to the model: hasStreaming checks whether the model returns a text/event-stream response for a streaming-enabled request; hasFunctions sends a prompt with a tool definition and validates that the model's response includes a structured tool call; hasVision submits a prompt containing a base64-encoded image and checks for coherent visual understanding in the response. Each probe is wrapped in its own timeout context (typically three to five seconds) so that a slow probe for one capability does not block the detection of others.

**TABLE 2: Feature Flags to HelixTranslate Capabilities**

| Feature Flag       | Bit Value | HelixTranslate Capability                                 | Fallback Behavior When Absent                         |
|--------------------|-----------|-----------------------------------------------------------|-------------------------------------------------------|
| CapStreaming       | 0x01      | Real-time translation with incremental output             | Buffer complete response before delivering to client  |
| CapFunctionCalling | 0x02      | Structured glossary/terminology injection via tools       | Prepend glossary terms to system prompt               |
| CapVision          | 0x04      | Translation of image-embedded documents (OCR + translate) | Reject image inputs with unsupported-media error      |
| CapCodeGeneration  | 0x08      | Context-aware code translation with syntax preservation   | Use generic text translation with code-postprocessing |
| CapJSONMode        | 0x10      | Structured output schemas (response format enforcement)   | Parse free-form response with JSON heuristics         |
| CapSystemPrompts   | 0x20      | Per-request behavior customization via system messages    | Embed instructions in the user prompt prefix          |
| CapLargeContext    | 0x40      | Document-level translation (>32K tokens input)            | Split document into chunks with overlap stitching     |
| CapMultilingual    | 0x80      | Single-model multi-directional translation                | Route through language-pair-specific model selection  |

Table 2 maps each feature flag to the corresponding HelixTranslate capability and describes the fallback strategy used when the flag is not set. This mapping is the bridge between raw model capability detection and actionable routing logic. For example, when `CapStreaming` is absent, the translation handler switches from a streaming response writer to a full-response buffer, delivering the complete translation only after the model finishes generation. When `CapVision` is absent, the API layer returns a `422 Unprocessable Entity` response with a descriptive error code, rather than attempting translation and producing garbled output. The fallback behaviors are implemented in the translation handlers themselves, which read the model's cached capability bitmap at request time.

Code Visibility evaluation assesses whether the model can accurately process and translate programming language content. This step is distinct from `CapCodeGeneration` because a model may support generating code (the ability to produce syntactically valid output) without correctly translating existing code comments, variable names, or string literals between natural languages. The Code Visibility step sends a curated set of code snippets containing translatable comments and documentation strings, then evaluates whether the model preserves syntax structure while accurately translating the natural-language content. The evaluation uses a combination of static analysis (AST parsing to verify structural integrity) and semantic similarity scoring (comparing translated comments against reference translations using embedding-based cosine similarity). Models that score below a configurable threshold on code visibility are still admitted to the pool but are excluded from the code-translation routing tier, ensuring that sensitive programming content is not routed to linguistically capable but code-naïve models.

---

## 3.5 Coding Challenge and Performance Benchmarking

Group 2 of the pipeline begins with two resource-intensive steps: the Coding Challenge and the Performance Benchmark. These steps are sequential because both generate load on the target model, and running them concurrently would skew latency measurements and potentially trigger rate limiting from the model provider.

The Coding Challenge step is a specialized variant of Code Visibility that focuses on active problem-solving rather than passive translation. It presents the model with a small set of well-known coding problems (e.g., implementing a specific algorithm, fixing a buggy function, or explaining a code snippet's behavior in another language) and evaluates the correctness of the response. This step serves two purposes: it validates that the model has retained its coding reasoning capabilities under the target inference configuration, and it produces a confidence score that feeds into the model's overall quality rating. The step is non-critical because a model that fails the coding challenge may still be perfectly adequate for general text translation, but the failure is recorded and used to downgrade the model's ranking in the selection algorithm.

The Performance Benchmark step is the longest-running step in the entire pipeline, with a timeout budget of 120 seconds. It measures the model's end-to-end translation latency across a representative sample set, computing percentile distributions and throughput metrics that directly inform routing decisions. The benchmark protocol follows a strict measurement discipline to ensure reproducible and comparable results across models and providers.

## CODE BLOCK 7: Performance Benchmark

```
func BenchmarkTranslation(ctx context.Context, modelID string,
    samples []string) (*PerfResult, error) {
    var latencies []time.Duration
    var tokens int
    for _, text := range samples {
        start := time.Now()
        result, err := client.Translate(ctx, modelID, text)
        if err != nil { return nil, err }
        latencies = append(latencies, time.Since(start))
        tokens += result.TokenCount
    }
    return &PerfResult{
        P50Latency: percentile(latencies, 0.50),
        P95Latency: percentile(latencies, 0.95),
        P99Latency: percentile(latencies, 0.99),
        TokensPerSec: float64(tokens) / totalTime.Seconds(),
    }, nil
}
```

The `BenchmarkTranslation` function takes a context, a model identifier, and a slice of sample texts of varying lengths. It iterates over each sample, records the wall-clock latency of the translation call, and accumulates the total token count from the response metadata. The function returns a `PerfResult` struct containing the P50, P95, and P99 latency percentiles, along with the average tokens-per-second throughput. These metrics are computed using a standard percentile function that sorts the latency slice and indexes into the sorted array.

**TABLE 3: Benchmark Parameters**

| Parameter            | Value       | Description                                                                                                                              |
|----------------------|-------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Warm-up iterations   | 3           | Performed before measurement to populate caches and establish connection pools; results are discarded                                    |
| Sample texts         | 10          | Curated to cover varied lengths from 100 to 5,000 characters, spanning short phrases, paragraphs, and multi-paragraph documents          |
| Measurement duration | 60s maximum | Hard cap per model to prevent benchmark runs from monopolizing the verification worker; early termination if all samples complete sooner |
| P95 TTFT target      | < 5s        | Time-to-first-token target; models exceeding this threshold receive a latency penalty in the scoring algorithm                           |
| P50 TPS target       | > 10        | Median tokens-per-second target; models below this threshold are flagged as low-throughput and deprioritized in routing                  |

Table 3 enumerates the benchmark's measurement parameters. The warm-up iterations are critical for fairness: without them, cold-start models (or models accessed over freshly established TLS connections) would report artificially high latencies that do not reflect steady-state performance. The ten sample texts are

drawn from a curated corpus stored in `internal/verifier/benchmark_samples.json`, which includes short phrases (100 characters), standard paragraphs (1,000 characters), and long-form documents (5,000 characters) across multiple languages and domains. This diversity ensures that the benchmark score reflects real-world translation workloads rather than a single text length. The 60-second measurement cap prevents a single slow model from blocking the verification pipeline indefinitely; if all samples are not processed within the cap, partial results are extrapolated with a warning annotation. The P95 TTFT and P50 TPS targets are service-level objectives that feed into the admission scoring: a model meeting both targets receives a latency bonus, while a model missing either receives a penalty that affects its position in the routing priority queue.

The `PerfResult` is persisted alongside the model's record in the verification cache, where it is refreshed on each scheduled verification cycle. Translation request handlers query this cached performance data when selecting a model from the active pool, preferring models with lower P95 latency for latency-sensitive requests and models with higher TPS for throughput-oriented batch jobs.

---

## 3.6 Cost Analysis and Final Validation

The final two steps of the pipeline transition from technical measurement to economic assessment and composite decision-making. Cost Analysis evaluates the financial efficiency of using the model for translation workloads, while Final Validation aggregates all preceding step results into a definitive admission verdict.

Cost analysis operates on the pricing data exposed by each model provider, which typically reports costs as separate rates for input tokens and output tokens. Because different providers use different pricing granularities (per 1K tokens, per 1M tokens, per character) and currencies, a normalization step is required before costs can be compared across the model pool.

### CODE BLOCK 8: Cost Normalization and Final Validation

```
func NormalizeCost(providerID string, inputCost, outputCost
    float64) float64 {
    // Normalize to cost per 1M tokens across all providers
    total := inputCost + outputCost*2 // weight output higher
    return 10.0 * math.Exp(-total/
        10.0) // exponential decay score 0-10
}

func FinalValidation(results []*StepResult) *VerificationReport {
    passed := 0
    for _, r := range results { if r.Passed { passed++ } }
    return &VerificationReport{
        TotalSteps: len(results),
        Passed: passed,
        Failed: len(results) - passed,
        OverallPass: passed >= 6, // 75% threshold
    }
}
```

The `NormalizeCost` function converts raw pricing data into a dimensionless score on a 0-to-10 scale, where 10 represents the most cost-effective model and 0 represents the most expensive. The normalization formula uses an exponential decay:  $10.0 * \text{math.Exp}(-\text{total}/10.0)$ , where `total` is the weighted sum of input and output costs. Output tokens are weighted at 2x relative to input tokens because translation workloads typically generate output that is comparable in length to the input, and in many language pairs the output is longer due to morphological expansion. The exponential decay ensures that the score remains bounded and that small differences in low-cost models are amplified while large differences among high-cost models are compressed. For example, a model costing \$0.50 per 1M tokens receives a score near 9.5, while a model costing \$20.00 per 1M tokens receives a score near 1.4. This score is stored in the model's metadata and used by the cost-aware router to prefer economical models when quality differences are negligible.

Cost analysis also accounts for provider-specific billing models. Some providers charge for API calls in addition to token consumption; others offer volume discounts or reserved throughput pricing. The cost step queries the verifier service's pricing endpoint, which maintains a normalized pricing database across all supported providers. The step's timeout is intentionally short (10 seconds) because pricing data is cached at the verifier level and should not require live API calls.

The Final Validation step is the pipeline's terminal node. It receives the slice of all `StepResult` values produced by the preceding seven steps and computes the aggregate `VerificationReport`. The report contains the total number of steps, the count of passed and failed steps, and a boolean `OverallPass` field that applies the admission threshold. The default threshold requires at least six of eight steps to pass, corresponding to a 75% success rate. This threshold is configurable via `LLMsVerifierConfig.MinPassingSteps` and can be tightened or relaxed based on operational requirements.

The threshold logic is intentionally lenient on non-critical steps. Because only three steps are marked as critical (Existence, Responsiveness, and Final Validation itself), a model can fail up to two non-critical steps and still achieve overall passage. For example, a model that passes Existence and Responsiveness but fails Feature Detection (because it lacks streaming support) and Code Visibility (because it performs poorly on code translation) would score six of eight passes, clearing the threshold and being admitted as a general-purpose text translation model with noted limitations. Conversely, a model that passes all non-critical steps but fails either Existence or Responsiveness is automatically rejected regardless of its other scores, because a non-existent or unreachable model cannot perform any translation.

The `VerificationReport` is the return value of `VerificationPipeline.Run` and is consumed by two downstream systems. First, it is written to the model cache, where the `OverallPass` flag determines whether the model is included in the active translation pool. Second, it is emitted as a structured log event and a Prometheus metric, enabling operations dashboards to track verification success rates, per-step failure frequencies, and model pool health over time. When a model transitions from passing to failing (or vice versa) between verification cycles, an alert is triggered so that operators can investigate provider outages, API changes, or model deprecation events.

The wiring of the entire Phase 2 engine into HelixTranslate is completed in `cmd/helix-translate/main.go`. After the configuration is loaded and the verifier client is initialized (as described in Section 3.1), the `VerificationPipeline` is

constructed with its eight steps populated from the handler files in `internal/verifier/`. The pipeline is then passed to a background verification worker that runs on a configurable interval (default: five minutes), cycling through all registered models and refreshing their verification reports. The worker uses a bounded goroutine pool to control concurrency, ensuring that the verification load does not overwhelm either the HelixTranslate host or the upstream model providers. On each cycle, models with updated reports are re-evaluated for pool membership, and the translation router is notified of any changes to the active model set.

Phase 2 thus delivers a fully autonomous verification subsystem that continuously assesses model health, capability, performance, and cost, making dynamic admission decisions without human intervention. The combination of parallel fast checks, sequential deep checks, retry resilience, and composite scoring provides a robust foundation for the intelligent routing layer that will be built in Phase 3.